

```
class Banner
  attr_accessible :horiz, :link, :visible, :image, :position
  has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
ng"===f.type(a)},F=function(a){return p(a)&&0<a.indexOf("%")},l=function(a,d){var
t(a,10)||0;d&&F(a)&&(e*=b.getViewport()[d]/100);return Math.ceil(e)},x=function(a,b){return
x"};f.extend(b,{version:"2.1.4",defaults:{padding:15;margin:20,width:800,
0,minWidth:100,minHeight:100,maxWidth:9999,maxHeight:9999,autoSize:!0,autoHeight:!1,autoWidth:!1,autoResize
enter:!s,fitToView:!0,aspectRatio:!1,topRatio:0.5,leftRatio:0.5,scrolling:"auto",wrapCSS:"",arrows:!0,close
seClick:!1,nextClick:!1,mouseWheel:!0,autoplay:!1,playSpeed:3E3,preload:3,modal:!1,loop:!0,ajax:{dataType:
ders:{"X-fancyBox":!0}},iframe:{scrolling:"auto",preload:!0},swf:{wmode:"transparent",allowfullscreen:"tru
riptaccess:"always"},keys:{next:{13:"left",
:"left",40:"up"},prev:{8:"right",33:"down",37:"right",38:"down"},close:[27],play:[32],toggle:[70]},directi
Left",prev:"right"},scrollOutside:!0,index:0,type:null,href:null,content:null,title:null,tpl:{wrap:'<div
ybox-wrap" tabIndex="-1"><div class="fancybox-skin"><div class="fancybox-outer"><div
ybox-inner"></div></div></div></div>',image:' {
    e.preventDefault();

    try {
      const res = await fetch('/api/login', {
```

```
method: 'POST',
headers: {
  'Content-Type': 'application/json',
},
body: JSON.stringify({ name, password }),
});
const data = await res.json();
console.log(res)
console.log(data)
if (!res.ok) {
  toast.error(`Erro ao efetuar login como usuário`, {
    position: "bottom-center",
    autoClose: 5000,
    hideProgressBar: true,
    closeOnClick: true,
    pauseOnHover: true,
    draggable: true,
    progress: undefined,
    theme: "colored",
    transition: Bounce,
  });
  throw new Error(data.message || 'Algo deu errado na autenticação');
}
document.cookie = `username=${data.name}; expires=Thu, 18 Dec 2029
12:00:00 UTC; path=/`;
```

```

document.cookie = `id=${data.id}; expires=Thu, 18 Dec 2029 12:00:00 UTC;
path=/`;

router.push('/persons')
} catch (error) {
}
}

return (
  <main className='flex flex-1 flex-col h-screen w-screen justify-center items-
center bg-background-900'>

    <p className='font-sans text-2xl text-fontColor-900'>FIAPCHAT</p>

    <div className='flex flex-1 flex-col max-h-[45vh] w-[80vw] justify-center items-
center bg-background-800'>

      <p className='font-sans text-1xl text-fontColor-900'>Digite um nome de
usuário</p>

      <input

        className='font-sans text-lg text-fontColor-900 bg-background-900 border
border-fontColor-900 rounded-xl p-2 w-5/6'

        type="text"

        placeholder=""

        onChange={(e) => setName(e.target.value)}/>

      <p className='font-sans mt-2 text-1xl text-fontColor-900'>Digite sua
senha</p>

      <input

        className='font-sans text-lg text-fontColor-900 bg-background-900 border
border-fontColor-900 rounded-xl p-2 w-5/6'

        type="password"

        placeholder=""

```

```

        onChange={(e) => setPassword(e.target.value)}/>

        <div className="flex flex-1 flex-row mt-2 w-[40%] max-h-12 ml-auto mr-auto
justify-between items-center">

            <Link href="/register" className='w-[max-content] p-2 font-sans bg-
fontColor-900 rounded-xl hover:opacity-80'>Não possui cadastro? Cadastre-se
aqui!</Link>

            <button disabled={{password == " " || name == ""}}onClick={handleLogin}
className='w-32 font-sans p-2 bg-fontColor-900 rounded-xl hover:opacity-80
hover:cursor-pointer'>Entrar</button>

        </div>
    </div>

    <ToastContainer
        position="bottom-center"
        autoClose={5000}
        hideProgressBar
        newestOnTop={false}
        closeOnClick
        rtl={false}
        pauseOnFocusLoss
        draggable
        pauseOnHover
        theme="colored"
        transition={Bounce}
    />

</main>

);
}

```

index.tsx (dentro da pasta persons)

```
import { UserCard } from "../../app/components/UserCard";

export async function getServerSideProps() {
  try {
    const response = await fetch('http://localhost:3333/api/users', {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
      },
    });
    if (response.status === 200) {
      const listaUsuarios = await response.json();
      return { props: { listaUsuarios } };
    } else {
      return { props: { listaUsuarios: [] } };
    }
  } catch (error) {
    console.error('Erro ao buscar dados:', error);
    return { props: { listaUsuarios: [] } };
  }
}

export default function Persons({ listaUsuarios }) {
  return (
```



```

<main className='flex flex-1 flex-col h-screen w-screen justify-center items-
center bg-background-900'>

  <p className='font-sans text-2xl text-fontColor-900'>listar as pessoas</p>

  {listaUsuarios.map((item:any) => (

    <UserCard key={item.id} data={item}/>

  )))}

</main>

);
}

```

#### UserCard.tsx

```

import { MessageSquareMore } from 'lucide-react';
import { useRouter } from 'next/router';

export const UserCard = ({ data }: { data: any }) => {

  const router = useRouter()

  const handleClick = () => {

    router.push({

      pathname: `/chat/${data.name}`,

      query: { recipientId: data.id },

    });

  };

  return(

```

```

<div className='flex flex-1 flex-row font-sans justify-around items-center bg-
background-800 w-3/4 max-h-10 mt-2 mb-2 rounded-xl hover:opacity-80 cursor-
pointer'

  onClick={handleClick}>

    <span className='flex flex-1 ml-10 w-2/4'>{data.name}</span>

    <div className="flex flex-1 justify-center w-2/4">

      <MessageSquareMore className='h-8 w-8 text-fontColor-900'/>

    </div>

  </div>

)
}

```

[destinatario].tsx

```

import { useEffect, useState } from 'react';
import { useRouter } from 'next/router';

export async function getServerSideProps() {
  try {
    const response = await fetch(`http://localhost:3333/api/users/id?id=${1}`, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
      },
    });
    if (response.status === 200) {

```

```

    const userSender = await response.json();

    return { props: { userSender } };

  } else {

    return { props: { listaUsuarios: [] } };

  }

} catch (error) {

  console.error('Erro ao buscar dados:', error);

  return { props: { userSender: null } };

}

}

export default function ChatPage({userSender}) {

  console.log(userSender)

  const router = useRouter();

  const { recipientId } = router.query;

  const [userName, setUserName] = useState("");

  const [id, setId] = useState("");

  useEffect(() => {

    const getCookieValue = (name:any) => (

      document.cookie.split('; ').find(row => row.startsWith(`${name}=`)).split('=')[1]

    );

    setUserName(getCookieValue('username')?? "");

    setId(getCookieValue('id')?? "");

  }, []);

```

```
return (  
  <div>  
    <p>Tela do chat</p>  
    <p>Nome de Usuário: {userName}</p>  
    <p>Nome de Usuário Remetente: {userSender?.name}</p>  
    <p>ID: {id}</p>  
    <p>ID Remetente: {userSender?.id}</p>  
  </div>  
);  
}
```

Não deixe de praticar!

## SAIBA MAIS

O Next.js é um framework de JavaScript para React que oferece várias formas de renderização para otimizar o desempenho e a experiência do usuário de aplicações web. Duas dessas formas são o Client Side Rendering (CSR) e o Server Side Rendering (SSR).

### Client Side Rendering (CSR)

Neste método, o conteúdo da página é gerado no navegador do usuário (client-side). Quando a página é carregada pela primeira vez, o servidor envia uma página HTML vazia e, em seguida, o JavaScript carrega os dados e gera o conteúdo dinamicamente no navegador. Isso significa que a maior parte do trabalho de renderização é feita no lado do cliente.

#### Vantagens do CSR

- Interações ricas e responsivas: uma vez que a página inicial é carregada, as interações subsequentes podem ser mais rápidas, pois não requerem viagens adicionais ao servidor.
- Menos carga no servidor: como o navegador do usuário lida com a renderização, há menos carga sobre o servidor, o que pode ser benéfico para escalabilidade.

#### Desvantagens do CSR

- Tempo de carregamento inicial: pode haver um tempo de carregamento inicial mais longo, pois o navegador precisa carregar o JavaScript antes de poder renderizar a página.

### Server Side Rendering (SSR)

Neste método, o conteúdo da página é gerado no servidor. Quando um usuário solicita uma página, o servidor processa o pedido, renderiza o conteúdo HTML final e o envia ao navegador. Isso significa que o usuário vê o conteúdo completo sem ter que esperar que o JavaScript seja carregado e executado.

### **Vantagens do SSR**

- Melhor SEO (Search Engine Optimization): como o conteúdo é gerado no servidor e enviado ao navegador em forma de HTML completo, é mais fácil para os motores de busca rastrear e indexar o site.
- Tempo de carregamento inicial mais rápido: os usuários podem ver e interagir com o conteúdo mais rapidamente, pois não precisam esperar que o JavaScript seja carregado e executado.

### **Desvantagens do SSR**

- Carga no servidor: pode haver uma carga maior sobre o servidor, pois ele precisa renderizar as páginas para cada solicitação.
- Tempo de resposta: em alguns casos, o tempo de resposta pode ser mais lento, especialmente se o servidor estiver sobrecarregado ou se a renderização da página for complexa.

### **Por que utilizar CSR e SSR em Next.js?**

A escolha entre CSR e SSR em Next.js depende das necessidades específicas do projeto. O Next.js oferece suporte a ambos os métodos, permitindo que pessoas desenvolvedoras escolham a melhor abordagem para cada página ou componente da aplicação. Isso permite combinar os benefícios de cada método para otimizar a experiência do usuário, o desempenho da aplicação e a eficácia do SEO. Por exemplo, você pode usar SSR para as páginas iniciais para obter um carregamento rápido, e um bom SEO e CSR para interações dinâmicas e ricas em páginas subsequentes.

Alguns tópicos interessantes sobre o assunto são:

### **Renderização híbrida**

A renderização híbrida em Next.js é uma abordagem poderosa que combina as vantagens tanto do Server Side Rendering (SSR) quanto do Client Side Rendering (CSR), permitindo às pessoas que desenvolvem otimizar suas aplicações web para desempenho, SEO e experiência do usuário. Esta abordagem híbrida é um dos grandes diferenciais do Next.js, oferecendo flexibilidade sem precedentes na maneira como as páginas são servidas e renderizadas.

## O que é renderização híbrida?

Renderização híbrida refere-se à capacidade de uma aplicação web usar diferentes métodos de renderização dependendo das necessidades específicas de cada página ou componente. No contexto do Next.js, isso significa que você pode ter páginas que são pré-renderizadas no servidor (SSR), páginas geradas estaticamente (Static Site Generation - SSG) no momento da construção, e páginas que são renderizadas no lado do cliente (CSR) após a inicialização.

## Benefícios da renderização híbrida

- Otimização de desempenho: ao servir páginas estáticas geradas no momento da construção para conteúdo que não muda frequentemente (SSG) e usar SSR para páginas dinâmicas que precisam de dados atualizados em tempo real, você pode otimizar o tempo de carregamento e a interatividade do usuário.
- Melhoria do SEO: páginas renderizadas no servidor ou geradas estaticamente são mais facilmente indexáveis por mecanismos de busca, melhorando a visibilidade e o ranking do site.
- Experiência do usuário aprimorada: a renderização no lado do cliente pode ser utilizada para partes dinâmicas da aplicação, proporcionando uma experiência de usuário suave e interativa sem recarregar toda a página.
- Flexibilidade de desenvolvimento: profissionais de desenvolvimento podem escolher a melhor estratégia de renderização para cada parte da aplicação, equilibrando entre desempenho, SEO e complexidade de desenvolvimento.

## Como funciona

O Next.js simplifica a implementação da renderização híbrida por meio de suas APIs e convenções. Veja a seguir:

**SSR com `getServerSideProps`:** para renderizar páginas no lado do servidor em cada solicitação, garantindo conteúdo sempre atualizado.

**SSG com `getStaticProps` e `getStaticPaths`:** para gerar páginas estáticas no momento da construção. Essas páginas podem ser revalidadas com Incremental Static Regeneration (ISR), permitindo atualizações sem reconstruir todo o site.

**CSR:** após a página ser servida (seja por meio de SSR ou SSG), o Next.js pode hidratar a aplicação no cliente, permitindo a navegação entre páginas e interações sem recarregar a página, usando CSR para atualizar partes da página dinamicamente.

### Casos de uso

- Blogs e sites de notícias: utilizar SSG para artigos e posts que não mudam frequentemente, e SSR para seções dinâmicas como comentários ou notícias de última hora.
- E-commerce: usar SSR para páginas de produtos para garantir informações atualizadas e SEO otimizado, enquanto utiliza CSR para interações do usuário, como carrinho de compras e recomendações personalizadas.
- Aplicações web complexas: combinação de SSR para primeiras impressões rápidas e SEO com CSR para funcionalidades ricas e interativas, como dashboards e aplicativos de edição.

A renderização híbrida em Next.js representa uma abordagem moderna e eficaz para construir aplicações web, oferecendo o melhor dos dois mundos: desempenho otimizado e uma excelente experiência do usuário, mantendo uma boa visibilidade nos mecanismos de busca.

### Dados e fetching

No desenvolvimento de aplicações web com Next.js, a maneira como os dados são buscados e gerenciados é fundamental para otimizar a experiência do usuário, o desempenho da aplicação e a otimização para motores de busca (SEO). Next.js oferece várias estratégias para buscar dados, seja por meio de Client Side Rendering (CSR) ou Server Side Rendering (SSR), cada uma com suas particularidades e melhores usos.



## Dados e Fetching em CSR

Client Side Rendering (CSR) refere-se ao processo de buscar dados e renderizar a interface do usuário diretamente no navegador do cliente, após o carregamento inicial do aplicativo. Em Next.js, o CSR é comumente utilizado em conjunto com hooks como `useEffect` para buscar dados de APIs ou outros serviços assim que o componente é montado no DOM.

### Vantagens

- Interações dinâmicas: permite atualizar a UI em tempo real sem necessidade de recarregar a página.
- Redução da carga no servidor: o servidor não precisa processar cada requisição para renderizar o HTML, reduzindo a carga e potencialmente aumentando a escalabilidade da aplicação.

### Desvantagens

- SEO: conteúdo carregado dinamicamente pode não ser indexado corretamente pelos motores de busca, afetando a visibilidade da página.
- Tempo de carregamento inicial: o conteúdo específico da página só é carregado e renderizado após a execução do JavaScript, o que pode resultar em uma percepção de lentidão pelo usuário.

## Dados e Fetching em SSR

Server Side Rendering (SSR) envolve a renderização do HTML no servidor com base nos dados solicitados, antes de enviar a página completamente renderizada ao navegador do cliente. Em Next.js, isso é frequentemente realizado utilizando `getServerSideProps`, uma função que roda no servidor para cada requisição, buscando dados necessários antes da renderização da página.

### Vantagens

- Melhor SEO: como o HTML é pré-renderizado no servidor, os motores de busca podem facilmente rastrear e indexar o conteúdo da página.

- Carregamento rápido: a página é enviada ao cliente já com o conteúdo renderizado, melhorando a percepção de desempenho e a experiência do usuário.

### **Desvantagens**

- Carga no servidor: cada requisição ao servidor resulta em uma nova renderização, o que pode aumentar a carga sobre o servidor, especialmente em sites com alto tráfego.
- Tempo de resposta: dependendo da complexidade dos dados a serem buscados e renderizados, o SSR pode aumentar o tempo de resposta do servidor.

### **Escolhendo entre CSR e SSR para dados e fetching**

A escolha entre usar CSR ou SSR em Next.js para dados e fetching depende de vários fatores, incluindo os requisitos de SEO, a natureza dinâmica do conteúdo, a necessidade de interações em tempo real e a experiência do usuário desejada. Algumas recomendações gerais incluem:

- Use SSR para conteúdo crítico: páginas que precisam de bom SEO ou que apresentam conteúdo crucial para o usuário devem preferir o SSR, garantindo que o conteúdo esteja disponível imediatamente após o carregamento.
- Utilize CSR para dados dinâmicos e interações: para áreas da aplicação que dependem de interações do usuário ou dados que mudam frequentemente, o CSR pode oferecer uma experiência mais fluida e interativa.

Além disso, Next.js também suporta Static Site Generation (SSG) e Incremental Static Regeneration (ISR), que podem ser utilizadas para otimizar ainda mais o desempenho e a experiência do usuário, permitindo a geração de páginas estáticas no momento da construção e sua atualização incremental.

Em resumo, a decisão entre CSR e SSR no contexto de dados e fetching em Next.js deve ser informada pela necessidade de balancear SEO, desempenho, carga

no servidor e experiência do usuário, aproveitando a flexibilidade que o Next.js oferece para escolher a abordagem mais adequada para cada caso.

## **Otimização de desempenho**

A otimização de desempenho é um aspecto crucial no desenvolvimento de aplicações web modernas, especialmente em frameworks avançados como Next.js, que suporta tanto Client Side Rendering (CSR) quanto Server Side Rendering (SSR). Entender como otimizar o desempenho utilizando essas duas abordagens pode melhorar a experiência do usuário significativamente, a escalabilidade da aplicação e sua visibilidade nos motores de busca. Aqui, exploraremos estratégias de otimização de desempenho específicas para o uso de CSR e SSR em Next.js.

### **Otimização no uso de SSR**

#### **1. Minimizar o tempo de resposta do servidor**

- **Caching de dados:** implementar cache para resultados de APIs ou consultas de banco de dados pode reduzir drasticamente o tempo de resposta do servidor, melhorando o desempenho do SSR.
- **Code Splitting:** utilizar a funcionalidade de code splitting do Next.js para carregar apenas o código necessário para renderizar a vista atual, reduzindo o tempo de carregamento inicial.

#### **2. SSR apenas quando necessário**

- **Avaliar páginas** que realmente se beneficiam do SSR, como aquelas importantes para SEO ou que necessitam de conteúdo dinâmico pré-carregado. Páginas estáticas ou dinâmicas que não precisam de indexação podem ser servidas via CSR ou SSG.

#### **3. Otimização de recursos**

- **Assegurar** que imagens e outros recursos estáticos sejam otimizados em tamanho e formato. Utilizar lazy loading para imagens e outros recursos pesados, carregando-os apenas quando estão prestes a entrar na viewport.

## **Otimização no Uso de CSR**

### **1. Carregamento assíncrono de dados e componentes**

Utilizar `React.lazy` para componentes e `Suspense` para dividir o código da aplicação e carregar partes da página sob demanda, melhorando o tempo de carregamento inicial e a interatividade.

### **2. State Management eficiente**

Gerenciar o estado da aplicação de forma eficiente, evitando re-renderizações desnecessárias. Ferramentas como `Redux`, `Zustand`, ou o próprio `Context API` do `React` podem ser utilizadas para gerenciar estados globais de forma otimizada.

### **3. Estratégias de fetching de dados**

Implementar estratégias inteligentes de fetching de dados, como `prefetching`, onde dados para outras páginas são pré-carregados antes da navegação do usuário, ou `caching` de dados no lado do cliente para reduzir requisições redundantes.

## **Otimizações gerais em Next.js**

### **1. Image Component e Image Optimization**

Utilizar o componente `Image` do `Next.js` que suporta otimização automática de imagens, carregamento `lazy` por padrão, e adaptação às dimensões da tela do usuário.

### **2. Incremental Static Regeneration (ISR)**

Para conteúdos que não mudam a cada requisição, mas que precisam de atualizações e requerem a reconstrução de toda a aplicação.

### **3. Análise e monitoramento de desempenho**

Utilizar ferramentas como `Google Lighthouse`, `Web Vitals` ou o próprio analisador de pacotes do `Next.js` para monitorar e analisar o desempenho da aplicação, identificando gargalos e áreas para otimização.

### **4. CDN e Edge Networks**

Servir conteúdo estático e páginas via `CDN` pode reduzir drasticamente tempos de carregamento ao diminuir a distância física entre os dados e o usuário final.

Ao aplicar essas estratégias de otimização, é possível tirar o máximo proveito das capacidades de CSR e SSR em Next.js, melhorando tanto a experiência do usuário quanto a eficiência operacional da aplicação. A chave é encontrar o equilíbrio entre renderização no lado do cliente e do servidor, alinhado com os objetivos específicos do projeto.

### **Escalabilidade**

Considerações sobre escalabilidade são cruciais, especialmente para aplicações de grande porte. SSR pode aumentar a carga no servidor, enquanto CSR pode exigir mais do dispositivo do usuário. Uma abordagem híbrida pode oferecer um equilíbrio eficaz.

## O QUE VOCÊ VIU NESTA AULA?

Você aprendeu como funciona a renderização em projetos Next.js, explorando três abordagens diferentes: CSR, SSR e Hybrid render. Também vimos boas práticas, vantagens e desvantagens destes métodos e como podemos utilizá-los em nossa aplicação.

Inicialmente, revisitamos um componente criado anteriormente e vimos qual método ele estava utilizando para renderização, que no caso era o Cliente Side Rendering, e posteriormente implementamos o Server Side Rendering em outro componente. Utilizamos um dos métodos nativos do Next e finalizamos usando o Hybrid Rendering em nossa aplicação, onde disponibilizamos um método versátil utilizando os recursos de CSR e SSR.

## REFERÊNCIAS

NEXT.JS. **Documentação Oficial**. [s.d.]. Disponível em: <https://nextjs.org/docs/>.

Acesso em: 28 jun. 2024.

OSMANY, A; MILLER, J. **Renderização na web**. [s.d.]. Disponível em:

<https://web.dev/articles/rendering-on-the-web?hl=pt-br>. Acesso em: 28 jun. 2024.

VERCEL. **Vercel**. [s.d.]. Disponível em: <https://vercel.com/blog/>. Acesso em: 28 jun.

2024.

EMANIP

## **PALAVRAS-CHAVE**

NextJs. SSR. CSR.

EMEND



# POSTECH